

LAB1

Name1: Mo Kong

Email1: mkong@kth.se

Name2: None

Submission ID: 20485310

Score: 23

J1

Central idea: My agent chooses the green boat's move with minimax and alpha-beta pruning, and also applied iterative deepening, move ordering and a transposition table.

Flow from input to output:

- Input: player_loop receive the game message -> Node(message) holding the hooks, fish, scores and the remaining observation sequence.
- Search: search_best_next_move runs iterative deepening on current root: depth 1, 2, 3,...until the time budget ends. Each iteration calls _search_root, and it calls the recursive _alphabeta on every root child. _alphabeta scores scores terminal states with _utility and depth-limit states with _evaluate. Otherwise it checks the transposition table(_state_key), expands children with compute_and_get_children, and searches the stored best move first(_order_children). The table is cleared every move but shared by its iterations.
- Output: The best move of the deepest finished iteration is returned

J2

Q1: A state holds the player to move, both scores, both hooks, the fish on each line, and the positions and values of the remaining fish. The observation file sets the initial hooks, fish, fish values and each fish's movement sequence. The successor function applies one of five actions (only "up" with a fish hooked), moves the fish by the next observation, updates catches and scores, and switches player.

Q2: A state is terminal when no fish is left or the observation sequence is exhausted. For green, the utility is the final score difference: positive is a win, negative a loss, zero a tie.

Q3: At a terminal state it equals the utility, so cut-off estimates and exact values are comparable; landed points cannot be lost, so a lead is real progress; and it costs $O(1)$, which matters because it is computed at every leaf.

Q4: Near the end, when few fish or steps remain, little can change. Early on it can say nothing: at the start of test_0 all five root moves evaluated to 0, because landing the nearest 11-point fish takes about 15 plies, beyond the search depth.

Q5: Green leads 2:0, so $v = +2$. It is red's turn and the only fish left, worth 10, is on red's line at $y = 18$. Red moves up, lands it and wins 2:10. v ignores the hooked fish.

Q6: η counts reachable wins and losses but ignores who chooses. Example: A's only move leads to s , where B chooses among three A-wins and one A-loss. $\eta(A,s) = 3 - 1 = 2 > 0$, but a rational B picks the loss, so A cannot force a win.

J3

Decision 1: Searching deeper

Goal: Search deep enough to see catches, without exceeding 75 ms per move.

Alternatives and Evidence: minimax with a depth-3 cut-off scored 12, and adding alpha-beta still gave 12: pruning only skips irrelevant branches, and a fixed depth must be safe for the slowest position, so the search stayed far below the roughly 15 plies needed to land a fish.

Choice and reason: From Section 2.5 I chose iterative deepening together with move ordering. Iterative deepening searches depth until time budget ends, so easy positions are searched deeper and there is always a finished answer. Each finished iteration also tells the next one which move was best, and searching that move first lets alpha-beta prune earlier. The score rose to 18. I then added a transposition table, cleared at every move. Each entry stores the depth searched, the value, whether it is exact or only a bound, and the best move. A repeated state is answered from the table when its entry was searched deep enough; otherwise its stored best move is searched first. The score rose to 22 (J4).

Code location: `search_best_next_move`, `_search_root`,
`_order_root_children`, `SearchTimeout`, `TIME_BUDGET`

Limitation: I did not try more complex techniques such as quiescence search, and the 55 ms budget is an untuned safety margin below the 75 ms limit.

Decision2: Evaluation function

Goal: make the agent act even when no point can be scored within the search horizon.

Failure case: After I got score 18 by applying iterative deepening, I found the green boat stayed still in the GUI in test_0. In test_0 the nearest valuable fish needs about 15 plies to be landed, deeper than the search, so the score difference was 0 for every move and ties went to "stay". And when a fish could be landed either now or a few moves later with the same final score, the search saw no difference, so it kept delaying.

Choice and reason: `_evaluate` now adds two terms to the score difference: fish already hooked, at 0.9 of their value because they are almost certain to be landed, and proximity to the best positive fish, which gives the hook a direction. A small bonus for scoring earlier breaks the ties behind procrastination. I rejected "never stay on a tie", because staying is correct when only negative fish remain.

Evidence: the boat started moving towards fish, even deep ones, and Kattis rose from 18 to 19. With the table in place, raising `PROXIMITY_WEIGHT` from 0.03 to 0.3 and `EARLY_BONUS` from 0.003 to 0.03 improved test_0 from 24:12 to 26:10, and gave 22–23 on Kattis. `EARLY_BONUS` = 0.05 gave 22, perhaps because it then rushes for small nearby fish.

Code location: `_evaluate`, `_proximity`, `_early_bonus` and their weight constants.

Limitation. the weights come from reasoning and a few submissions, not systematic tuning; both changed together, so the gain cannot be attributed to either; and one point may be noise.

J4

Advanced analysis: transposition table

How it works: The table maps a state key(`self._table`) to the depth searched, the value, the value type and the best move. The key holds everything that affects the search below a node: the depth, which is the position in the observation sequence; the player to move; both hooks; the hooked fish; the score margin, because values are measured relative to it; and the remaining fish. The table is cleared every move, since depth is counted from each move's root, but shared by all iterations of that move.

It is used in two ways:

- Value reuse: after searching a node, `_alphabeta` stores the value as `EXACT`, or as `LOWER/UPPER` if alpha-beta cut it off and it is only a bound. A stored value is returned directly only if it was searched at least as deep as now required and, for a bound, already decides the node for the current window.
- Ordering: otherwise the stored best move (`tt_move`) is searched first (`_order_children`). My earlier move ordering (`_order_root_children`) only reordered the five root moves using the previous iteration's answer; the table applies the same idea at every node searched before.

Why it helps: Moves commute (up-then-left reaches the same state as left-then-up) and the fish follow a fixed sequence, so many paths reach the same state.

Evidence (Kattis). No table: 19. Ordering only: 20. Value reuse only: 22. Both: 22, sometimes 23. Value reuse gives most of the gain. Ordering helps on its own, but adds little once values are reused; the occasional 23 is within the variation of a time-limited search. I kept both.

Limitation and alternative: Building a key costs time proportional to the number of fish at every node. Keeping the table between moves instead of depth, would reuse more work but was not tried.

J5

Contribution: I worked alone. I chose which techniques to add and in what order, tested in the GUI and located the problems in the code, and designed comparison experiments.

Understanding: I expected alpha-beta alone to raise the score, but with a fixed depth its saved time went unused. Iterative deepening turns saved time into depth, and repeating shallow iterations is cheap because each extra ply multiplies the tree.

Hardest Issue: the boat staying still. I first thought my search code was broken, but every root move had the same value: the nearest fish was beyond the search depth, so the score difference was 0 everywhere and ties went to the first child, "stay". This led to the evaluation changes in J3.

Open question: the same code scores 22 or 23, so small improvements are hard to separate from noise.

J6

I used Claude throughout the assignment, for four purposes:

1. **Explaining** the assignment and the algorithms (minimax, alpha-beta, iterative deepening, transposition tables).
2. **Implementing the code step by step.** I chose to start from the simplest version, minimax with a depth cut-off, and improve it one technique at a time. At each step I decided what to add next, and the AI wrote most of the code to my requirements and explained it until I could explain each part myself. I tested every version in the local GUI and on Kattis, adjusted the evaluation weights, and ran the comparisons in J3 and J4.
3. **Adding comments** to the code.
4. **Assisting with the journal:** Translating and polishing the content I wrote.

What I rejected or corrected. The AI proposed some solutions that were too complex, such as a prediction-based timer, which I rejected; I also asked for simpler garbage-collection handling. I noticed in the GUI that the boat don't moved, which led to the evaluation changes in J3.